

Especificação SDD TaskAnalyzer - Giovani Silva Rodrigues

1. VISÃO GERAL E CONTRATO DE NEGÓCIO (SDD)

Identificação:

- **Nome completo:** Giovani Silva Rodrigues
- **Curso:** Análise e Desenvolvimento de Sistemas
- **Polo/Turma:** UniCEUB Asa Norte - Brasília, DF
- **E-mail institucional:** giovani.rodrigues@sempreceub.br

Propósito do Módulo TaskAnalyzer:

O módulo é uma biblioteca de análise de produtividade projetada para processar coleções de tarefas operacionais, calcular o tempo médio de conclusão (em dias/horas), computar a taxa percentual de tarefas entregues em atraso em relação ao prazo estipulado e segmentar esses indicadores por nível de prioridade (baixa, média e alta). Atua como motor determinístico para apoio à tomada de decisão gerencial.

Contrato Executável de Interface:

Entradas: TaskAnalyzer.analyze(tasks: list[dict]) -> dict

Campo	Tipo	Obrigatório	Regras de Validação
id	str int	Sim	Identificador exclusivo e não vazio.
titulo	str	Sim	Texto não vazio descrevendo a tarefa.
prioridade	str	Sim	Valor restrito estritamente a: "baixa", "media", "alta".

Campo	Tipo	Obrigatório	Regras de Validação
data_criacao	datetime str (ISO-8601)	Sim	Ponto temporal inicial do ciclo da tarefa.
prazo_limite	datetime str (ISO-8601)	Sim	Deve ser cronologicamente posterior ou igual a data_criacao.
data_conclusao	datetime str (ISO-8601) None	Não	Obrigatório se status == "concluida". Deve ser posterior ou igual a data_criacao.
status	str	Sim	Valor restrito estritamente a: "pendente", "concluida".

Saídas: Retorna um dict com estrutura padronizada contendo:

- total_geral: Total de tarefas analisadas.
- total_concluidas: Contagem de tarefas com status "concluida".
- tempo_medio_conclusao_horas: Média de tempo decorrido entre criação e conclusão (considerando apenas concluídas).
- taxa_atraso_percentual: Percentual de tarefas concluídas com data_conclusao > prazo_limite.
- metricas_por_prioridade: Dicionário chaveado por prioridade ("baixa", "media", "alta"), contendo total, concluídas, tempo médio e taxa de atraso específicos.

Restrições de Negócio:

- Cálculos de produtividade, tempo decorrido e atraso aplicam-se exclusivamente a tarefas com status == "concluida".
- Em caso de ausência de tarefas concluídas, métricas agregadas devem evitar divisão por zero, retornando 0.0 ou disparando exceção específica tratada caso a lista esteja inteiramente inconsistente.

- Datas retroativas onde a conclusão anteceda a criação são inválidas e disparam `InvalidTaskDataError`.

2. ESPECIFICAÇÃO DE CENÁRIOS DE ACEITE E TEST HARNESS

Cenários de Teste (BDD / Dado-Quando-Então):

Cenário 1 (Sucesso - Métricas Corretas):

- **Dado** uma lista válida contendo 3 tarefas concluídas (1 com atraso, 2 no prazo) com prioridades alta e média,
- **Quando** o método `TaskAnalyzer.analyze(tasks)` for executado,
- **Então** a saída deve reportar `total_concluidas = 3`, `taxa_atraso_percentual = 33.33%`, o tempo médio global em horas calculado e o agrupamento coerente dentro de `metricas_por_prioridade`.

Cenário 2 (Exceção - Datas Inconsistentes):

- **Dado** uma lista contendo uma tarefa com `data_conclusao` anterior à sua `data_criacao`,
- **Quando** o método `TaskAnalyzer.analyze(tasks)` for executado,
- **Então** o sistema deve disparar a exceção customizada `InvalidTaskDataError` contendo a mensagem contextual descritiva e não retornar valores computados.

Cenário 3 (Exceção - Conjunto Sem Concluídas / Divisão por Zero):

- **Dado** uma lista vazia ou contendo apenas tarefas pendentes,
- **Quando** o método `TaskAnalyzer.analyze(tasks)` for executado com configuração de cálculo estrito,
- **Então** deve disparar `NoCompletedTasksError` ou consolidar métricas de média e taxa como 0.0 de forma segura sem falha de sistema.

Planejamento do Test Harness:

Os cenários acima serão convertidos em asserções automatizadas no `pytest` na Fase 2. Cada cenário se desdobrará em fixtures parametrizadas em `tests/test_harness.py`. O assistente de IA só terá seus artefatos aprovados se a suíte de testes passar com 100% de sucesso sem modificação dos testes pré-estabelecidos.

3. GOVERNANÇA DE CONTEXTO E REGRAS PARA AGENTES DE IA (CONTEXT_RULES)

Diretrizes Arquiteturais (Obrigatórias):

- Compatibilidade estrita com Python 3.11+.
- Uso integral de type hints em argumentos e retornos de classes/funções.
- Aderência irrestrita às convenções PEP 8 e princípio da responsabilidade única (SRP).

- Documentação via Google Style Docstrings em todos os módulos, classes e funções.
- Rastreabilidade via biblioteca padrão logging estruturada (níveis INFO/WARNING/ERROR).
- Exceções customizadas herdadas de Exception para cada caso de falha de validação.

Proibições Explícitas (Regras Restritivas):

- Não adicionar bibliotecas externas fora do requirements.txt básico (pytest).
- Proibida a persistência de dados em arquivos locais (JSON, TXT, SQLite) ou bancos de dados externos.
- Proibido alterar contratos de interface: nomes de métodos e parâmetros públicos são imutáveis.
- Proibido alterar, comentar ou enfraquecer asserções dentro de test_harness.py.
- Proibida a suposição de campos ou regras não documentadas na especificação SDD.

4. ARQUITETURA DO REPOSITÓRIO E PREPARAÇÃO PARA GIT/GITHUB

Árvore do Repositório (Fase 2):

```
sdd-task-analyzer/
├── README.md
├── CONTEXT_RULES.md
├── requirements.txt
├── .gitignore
├── specs/
│   └── task_analyzer_spec.md
├── tests/
│   └── test_harness.py
└── src/
    ├── __init__.py
    ├── exceptions.py
    └── task_analyzer.py
```

Plano de Homologação Humana:

1. **Passo 1 (Validação Funcional):** Execução do comando `pytest tests/` no ambiente local/container para verificar aprovação unânime das asserções.
2. **Passo 2 (Auditoria de Contrato e Regras):** Confronto do código gerado com o `CONTEXT_RULES.md` e verificação de integridade das assinaturas.
3. **Passo 3 (Qualidade de Código):** Inspeção manual de legibilidade, ausência de redundâncias e validação do padrão Google docstrings.
4. **Passo 4 (Testes Manuais de Borda):** Execução do analisador contra inputs extremos (ex: listas de 10.000 itens ou payloads com campos nulos opcionais).

5. **Passo 5 (Aprovação e Commit):** Realização do commit atômico e pull request apenas com a chancela do desenvolvedor.